Engine Numerical Loss Re-Rolls LLM Outputs but Shows No Detectable Increase in Failure Symptoms:

A Contract-Exact Baseline Study at 8-Bit Precision

Sunghyeun Woo
Chungwoon University, Republic of Korea

Draft for arXiv (cs.CL / cs.LG), October 2026

Engine and experiments: https://github.com/wwoosshh/GeulPU
Language and compiler: https://github.com/wwoosshh/geul-lang

Abstract

Large language models sometimes produce familiar failures: endless repetition, text in an unrelated script, broken spelling, wrong answers. One hypothesis attributes part of these failures to the numerical loss of inference engines—activations re-quantized to 8 bits, half-precision attention and caches, reduction orders that change with the batch. Testing it requires separating two effects that look alike in aggregate: loss that merely *re-rolls* near-tie decisions, and loss that systematically *tilts* the model toward abnormal tokens. We use GeulPU—an inference engine whose arithmetic follows a written precision contract, is bit-identical to a CPU reference, and produces identical bits across GPU generations and vendors—as a contract-exact, noise-free baseline, and compare it with llama.cpp and with a single-loss variant on Qwen2.5-7B-Instruct and DeepSeek-R1-Distill-Qwen-7B (Q8_0, greedy decoding, identical token inputs). On 696 Korean prompts and 600 math problems, llama.cpp changes 89–100% of the outputs, yet failures appear about as often as they disappear: text in another script 36 new vs. 35 gone, wrong or unfinished math answers 25 vs. 28, with 95% intervals that bound the net change to between -17% and $+19\%$ and between -22% and $+15\%$ of the baseline's counts. At 10,817 fragile steps—where an abnormal token competes closely with the chosen one—we find no tilt toward other-script or byte-split tokens (equivalence within ± 0.05 nat) and a small tilt toward repetition ($+0.0039$ nat, about 0.4% in odds) that is specific to repeating tokens; measured against fp64 arithmetic on a 0.5B model, the baseline's own rounding tilts repetition steps by only $+0.0005$ nat and other steps not at all. Under greedy decoding and contract-exact computation, most failures arise in the model itself (32% of long-form Korean answers loop to the length limit), and at steps where an other-script token is already a close competitor, forcing it is followed by more other-script text in 80% of continuations, against 34% after the original token. All hypotheses, statistics and decision rules were pre-registered.

1 Introduction

Users of large language models (LLMs) know a small set of recurring failures: the model repeats a phrase until it runs out of tokens, switches to an unrelated language or script in the middle of an answer, misspells a word it wrote correctly a moment ago, or loses the point of a problem. These failures are usually attributed to the model—its training data, its size, the decoding settings. A competing hypothesis is that part of them comes from the arithmetic of the inference engine. Production engines trade precision for speed in many places: llama.cpp re-quantizes activations to 8 bits before its quantized matrix multiplications, computes attention and stores the key–value cache in half precision, and, like most engines, changes the order of floating-point reductions with

the batch composition. Each of these choices perturbs the logits slightly, and a slightly different logit at a near-tie can send a generation down a different path.

That engine numerics change outputs is well established. Changing the batch size, the number of GPUs or the engine alone moves benchmark scores by several points (Yuan et al., 2025; Pape et al., 2026), prefix caching makes the same request diverge (Patodiya, 2026), and deterministic modes have been built to remove the variation on one machine (He et al., 2025; Qin et al., 2026). What is not established is whether this variation *degrades* the output—produces more failures—or only *re-rolls* it, turning some answers worse and as many better. The two are indistinguishable in a single run; they differ in the balance of failures that appear and disappear, and, at the level of single decisions, in whether the perturbation is biased toward abnormal tokens.

Measuring that bias needs a baseline without engine noise. If the reference itself is noisy, selecting the decisions where an abnormal token is close to the chosen one already depends on the noise, and the measured shift regresses toward zero. We therefore use as the baseline an engine whose loss is limited to what a written contract specifies and whose bits do not depend on batch, path, GPU or vendor (Section 3), feed identical token ids to every engine, and pre-register the analysis.

Our contributions are:

1. A paired design with a contract-exact baseline that separates *re-rolling* (symptoms appear and disappear in equal numbers) from *tilting* (a systematic shift toward abnormal tokens), with two pre-registered primary tests—trajectory-level McNemar tests with interval bounds on the net change, and a step-level tilt test at fragile steps with an equivalence band.

2. Evidence, for two 7B models at Q8_0 under greedy decoding, that llama.cpp's numerical loss re-rolls outputs without a detectable increase in failures—the net change is bounded within about ±20% of the baseline's symptom counts—and that its only detectable tilt is toward repetition, about 0.4% in odds.

3. Model-level measurements under contract-exact computation and greedy decoding: high failure base rates, the effect of a single other-script token at steps where it was already plausible, no effect of a single injected anomaly in the previous turn, and the fraction of Korean syllables that are emitted as byte pieces.

4. The instrument itself: an open inference engine with a written precision contract whose outputs are bit-identical across NVIDIA generations, AMD and Intel GPUs and a CPU reference (Section 3, Appendices A and D).

2 Related work

Engine nondeterminism and evaluation. He et al. (2025) trace run-to-run differences in served LLMs to batch-dependent kernels and provide batch-invariant kernels; vLLM and SGLang ship deterministic modes based on them. Vosti specifies deterministic inference formally and verifies an engine against the specification, showing that the production modes still diverge under some execution variations (Qin et al., 2026). Yuan et al. (2025) report accuracy swings of up to 9% for DeepSeek-R1-Distill-Qwen-7B across GPU counts, GPU types and batch sizes in bfloat16; Pape et al. (2026) find that the choice of engine alone moves benchmark scores by up to 16.6 points; Patodiya (2026) shows that llama.cpp's prefix cache makes repeated requests diverge, more so at 4 bits. These works establish variance; we ask whether the variance is biased.

Bit-exact inference across hardware. A CPU emulator can predict the bits of unmodified vLLM and Transformers runs on several NVIDIA GPUs, enabling verification without changing the engine (arXiv:2606.00279, 2026). Fixed-shape reduction orders make linear layers identical

across NVIDIA Ampere, Ada and Hopper (Cooper et al., 2026). Power-of-two weight scales make CUTLASS and Triton agree bitwise and expose the limits of tolerance-based conformance tests (Chen, 2026). Gensyn's REE reproduces inference across NVIDIA GPUs, Apple GPUs and CPUs with a proprietary operator library. To our knowledge no published system shows identical bits across NVIDIA and AMD GPUs with an open, written precision contract; we use such a system as an instrument rather than as the subject.

Failure modes. The Language Confusion Benchmark measures how often models answer in the wrong language (Marchisio et al., 2024a); quantization hurts non-Latin scripts most, and automatic metrics understate the harm (Marchisio et al., 2024b). Repetition is self-reinforcing—each repetition makes the next more likely (Xu et al., 2022)—and repetition requests can make production models diverge (Nasr et al., 2023). Under-trained "glitch" tokens trigger anomalous behaviour (Land and Bartolo, 2024). We are not aware of work that injects or removes individual sources of numerical loss and measures their effect on these failures with a contract-exact baseline.

3 The instrument: GeulPU

Our conclusions rest on two properties of the baseline: it is a complete inference engine that runs the same quantized model files as llama.cpp on the same GPU, and its arithmetic is exact with respect to a written contract. Both are unusual for software that is not built on Python, PyTorch or the CUDA toolkit, and the language it is written in is little known; Appendix D describes the language, the compiler and the engine from the bottom up, and Figure 1 there shows the stack. This section gives what the experiment relies on.

3.1 The engine in brief

GeulPU is written in geul (글), a systems language with Korean grammar whose self-hosted compiler has its own x86-64 back end and uses no C compiler or LLVM. A kernel generator, also in geul, writes model-specific GPU kernels; two back ends in geul translate them to PTX (NVIDIA) and SPIR-V (Vulkan: AMD, Intel, NVIDIA). The engine hands the PTX or SPIR-V to the vendor's driver and talks only to the driver: nothing from the CUDA toolkit or a Vulkan SDK is used to build or run it. The host—GGUF reader, tokenizers, scheduler, sampler and an OpenAI-compatible HTTP server with up to 8 slots—is about 13,000 lines of geul. It reads the same GGUF files as llama.cpp (Q8_0, Q4_0 and the k-quants; Qwen2.5, Qwen3, DeepSeek-R1-Distill-Qwen, Llama 3.1/3.2, Mistral), and on an RTX 4070 Ti decodes within ±5% of llama.cpp's speed at Q8_0 and Q4_K_M. The CPU reference of the contract, the test harnesses and the benchmarks are Python (NumPy).

3.2 The precision contract

GeulPU's arithmetic follows a written contract (`docs/contract.md`) that leaves the hardware no discretion:

- **Linear layers.** Each block of 32 activations is scaled by a power of two from its largest value and rounded to integers of at most 2^{14}, stored as two signed 8-bit digits. The products with the model's integer weights are summed exactly on int8 tensor cores (prompts) or with `dp4a` (decoding)—exact integer sums do not depend on summation order—then rounded once per block and accumulated with fused multiply–adds in a fixed order.
- **Attention.** Queries, keys and values are converted the same way and the key–value cache stores the digits; scores are exact integer sums rounded once; the exponential is a written polynomial; keys are processed in fixed 64-key chunks and folded in a fixed two-level order.

- **Everything else.** Normalizations use fixed reduction trees, rotary tables are correctly rounded, and one rule fixes how subnormal numbers flush on every path (PTX `.ftz`, the Vulkan drivers and the CPU reference agree on it); sampling and log-probabilities are computed in float64 in a fixed order.

Because nothing depends on how the work is split, the bits do not depend on batch, slot, chunking, prompt-versus-decode path, GPU or vendor, and they equal those of a CPU reference implementation of the contract in NumPy. We call such a baseline *contract-exact*: its only numerical loss is the one the contract writes down, and it is deterministic—but it is not exact arithmetic. The contract's loss is small: on Qwen2.5-0.5B (Q8_0) the Kullback–Leibler divergence of its next-token distributions from an fp64 computation is 2.24×10^{-6}, against 1.38×10^{-3} for llama.cpp; for 7B and 14B the corresponding ratios (1/3,805 and 1/4,467) come from a simulation of the rounding points rather than from the engine. Measured against fp64 arithmetic on Qwen2.5-0.5B (Appendix E), the contract's own rounding does not tilt other-script or byte-split steps and tilts repetition steps by +0.0005 nat—an order of magnitude below the effects measured in Section 5. Exactness has a price: at equal optimization, keeping two activation digits instead of one makes prompt processing 1.26–1.38× slower (a lower bound measured by switching the tile kernels), while decoding, which is memory-bound, is unaffected.

3.3 Bit identity

The contract's claims are checked on kernel test cases, on whole models end to end, across execution paths, and across machines: identical logit hashes at every step on NVIDIA Ampere, Ada and Blackwell GPUs through CUDA, on an NVIDIA and two AMD GPUs through Vulkan, and against the CPU reference (Appendix A). The vendors' closed compilers below PTX and SPIR-V are outside the contract, which is why bit identity is established empirically rather than proved.

4 Method

4.1 Arms and inputs

- **G (contract-exact baseline)**—GeulPU under its precision contract (Section 3): loss only where the contract writes it, no dependence on batch, slot or execution path. OpenAI-compatible server, 8 slots.
- **L (production engine)**—llama.cpp build b11669 (Windows CUDA 13.4), `llama-server` with default settings (flash attention *auto*, F16 key–value cache, prompt cache on), 8 parallel slots, the same GGUF files.
- **V (one loss)**—G with a single change: the input activations of all linear layers rounded to one digit instead of two—block grid 2^{E-5}, $|n| \le 64$, i.e. 7 bits per value with a power-of-two scale. llama.cpp's activation quantization (Q8_1: blocks of 32 scaled by max /127) uses 8 bits with an exact-maximum scale, so V's rounding step is 2–4× llama.cpp's: V is a coarser member of the same family, not a replica of llama.cpp's loss. Attention and everything else unchanged.

All prompts were rendered once with llama.cpp's `/apply-template` (the chat template stored in the GGUF) and tokenized with `/tokenize`; every arm received the same token ids. Decoding was greedy (temperature 0) with the same maximum length and end tokens, so the arms differ only in arithmetic. The context was 4,096 tokens per slot. Models: Qwen2.5-7B-Instruct (Korean prompts) and DeepSeek-R1-Distill-Qwen-7B (math; R1-Distill below), both Q8_0, so the weight loss is identical across arms and small.

Table 1: Prompt sets.

Set	Items	Max new	Model	Source
Natural: LCB Korean	396	1,024	Qwen2.5	Language Confusion Benchmark (Marchisio et al., 2024a), Korean native and cross-lingual prompts (Apache-2.0); 3 overlong prompts excluded
Natural: long-form Korean	300	2,048	Qwen2.5	60 topics × 5 forms ("write 2,000 characters or more")
Natural: math	600	3,200	R1-Distill	gsm8k test (Cobbe et al., 2021), first 600, with "Please reason step by step, and put your final answer within `\boxed{}`."
Trigger: rare syllables	200	512	Qwen2.5	"Copy the following text exactly": 80–200-character texts with ≥ 6 syllables that the tokenizer splits into byte pieces
Trigger: mixed script	200	1,024	Qwen2.5	Korean questions about Chinese/Japanese words and names written in their own script
Trigger: repetition	200	2,048	Qwen2.5	requests to repeat a word or sentence, count in Korean words, continue a pattern
Contamination dialogs	100 × 4	512 + 1,024	Qwen2.5	two-turn dialogs; the first answer is G's own, unchanged or with one injected anomaly

4.2 Prompt sets

Table 1 lists the prompt sets: natural prompts, and trigger sets built to make each symptom frequent enough to study.

4.3 Symptoms

All symptoms are decided automatically from token ids and text:

- **S1 endless repetition**—the generation hit the length limit and its tail is an exact repetition: the smallest period $p \leq 512$ tokens whose repetition covers at least $\max(3p, 64)$ final tokens.
- **S2 other-script text** (Korean sets)—S2a: a run of two or more letters that are neither Hangul nor Latin (Han, kana, Cyrillic, Thai, Arabic, Devanagari, ...; Greek excluded), outside code, not present in the question; or S2b: less than 50% of the letters are Hangul.
- **S3 unfinished / S4 wrong answer** (math)—S3: length limit without `\boxed{}`; S4: the last boxed number differs from the gold answer (S3 counts as wrong).
- **S6 broken spelling** (copy trigger)—the copied span differs from the source by at least one character, or contains a broken UTF-8 character.

4.4 Fragile steps and tilt

At each generated step of G, let a be the chosen token (greedy: G's top-1) and b the highest-ranked abnormal candidate of a given type among G's top 20:

- **O (other script)**—a is Hangul, Latin, digit or symbol; b contains a letter of another script (byte-piece tokens are classified by the script their lead byte implies);

- **R (repetition)**—appending a does not copy an earlier 8-token sequence of the answer, appending b does;
- **C (byte-split character)**—the step is inside a multi-byte character (earlier tokens left the UTF-8 sequence incomplete), and b completes a different character or breaks the encoding.

A step is *fragile* if the margin $m = \ln p_G(a) - \ln p_G(b)$ is below $\ln 10$. For an arm $X \in \{L, V\}$, the tilt at the step is

$$\Delta_X = \big[\ln p_X(b) - \ln p_X(a)\big] - \big[\ln p_G(b) - \ln p_G(a)\big], \tag{1}$$

computed on the same prefix (G's answer). Zero-mean numerical noise gives $\mathbb{E}[\Delta] = 0$; a systematic shift toward abnormal tokens gives $\Delta > 0$. As a control we compute the same quantity for the highest-ranked *normal* alternative at the same step. The mean Δ is estimated over all fragile steps with a 95% interval from a bootstrap that resamples prompts (10,000 resamples, seed 40); the three types are corrected with Holm's method. Decision labels (pre-registered): *tilts toward abnormal* (mean > 0, corrected $p < 0.05$), *tilts away*, *no tilt* (corrected $p \geq 0.05$ and the interval within ± 0.05 nat), *undecided* otherwise or with fewer than 200 steps.

To obtain L's distribution at G's prefixes we drive llama.cpp through its C API: the prompt is evaluated in one batch, as the server does, and G's answer tokens are then fed one at a time, eight sequences per batch. This reproduces one realization of llama.cpp's arithmetic—eight sequences of G's tokens per batch, which need not match the batch composition the server had while generating L's own answers—and we validated it against `llama-server` itself on the steps before the first divergence (Section 5.1). L's trajectories (Section 5.2) come from the server itself; only the step-level tilt uses this follower.

4.5 Trajectories, propagation and contamination

- **Trajectories (primary).** For each prompt, a symptom present only in the arm is *new*, present only in G *gone*. H1: Qwen2.5-7B, natural Korean sets, S2. H2: R1-Distill, math, S4. Exact McNemar tests, Holm over the two. Labels: *increases*, *decreases*, *shakes but does not increase* (new + gone ≥ 40, corrected $p \geq 0.05$), *undecided* (new + gone < 40; H2 would first add the next 600 problems once—not needed). We also report the 95% interval of the net change (new minus gone), from the binomial distribution of the discordant pairs, as the bound the data place on an increase or a decrease.
- **Propagation.** At up to 100 fragile steps per type (seed 40) we force b in G and continue for 256 tokens, and compare with G's own continuation after a: does a symptom of the same type appear again?
- **Contamination.** We insert one anomaly into G's first answer of a two-turn dialog—a Chinese word, a misspelled word, or a duplicated sentence—and compare the second answer with the one generated after the unchanged first answer.

4.6 Pre-registration

The design, hypotheses, statistics and decision rules were committed to the public repository at 09:28 on 2026-10-16 (commit `9190f76`); the first run started at 09:51. The prompt sets and the symptom judges were committed at 10:00, while G's Qwen2.5 run (09:59–10:39) was in progress; fixes to the tools (the end-token step, the BOS token, the context length, the determinism reruns) were committed at 10:44, after that run and before the V (from 12:19) and L (from 14:51) runs. No L or V data existed before 12:19, so no primary comparison could have been seen when the judges and tools were fixed. The registration lives in the author's repository rather than at an independent registry, and the whole study—registration, runs, analysis and this write-up—was completed within one day; deviations are listed in Appendix B.

Table 2: Primary trajectory tests, llama.cpp (L) against the contract-exact baseline (G). G / L: items with the symptom in each arm; new: items with the symptom only in L; gone: only in G; net change: new minus gone with its 95% interval from the binomial distribution of the discordant pairs, also as a fraction of G's count.

Test	Items	G / L	New	Gone	Net change, 95% CI	Holm p	Label
H1: Qwen2.5-7B, natural Korean, other script (S2)	696	94 / 95	36	35	−16 to +18 (−17% to +19%)	1.0	shakes but does not increase
H2: R1-Distill-7B, gsm8k, wrong or unfinished (S4)	600	80 / 77	25	28	−18 to +12 (−22% to +15%)	1.0	shakes but does not increase

5 Results

5.1 Validity checks

- **Identical inputs**: in every arm and item, the number of prompt tokens the engine received equals the number sent.
- **Determinism**: 50 prompts re-run at concurrency 1 give identical tokens in G and in V; 50 forced-token continuations re-run give identical tokens.
- **The L measurement path**: for 50 prompts generated by `llama-server` with one slot, the follower's argmax equals the server's token at all 3,358 steps up to the first divergence from G (on average 67 steps per prompt).
- **Human check of the symptom judges** (20 items per symptom: 10 judged positive, 10 negative): repetition 19/20, other-script 17/20—below the pre-registered 90%; with 20 items the 95% interval of the 85% agreement is about 62–97%. The other-script judge counted Han characters used to explain a Chinese word and English song titles quoted from the prompt. Because both arms of each comparison use the same judge, the paired tests are not biased toward either arm; a stricter post-hoc judge (Section 5.6) leaves every conclusion unchanged.

5.2 Trajectories: re-rolled, with no detectable increase

Table 2 gives the primary tests. Neither reached significance, and the intervals exclude large effects in either direction but not a moderate increase: the data are consistent with llama.cpp adding up to about a fifth more other-script answers, or removing about a sixth. llama.cpp produced a different text for 89% of the LCB prompts, all 300 long-form prompts and 97% of the math problems; the first divergence came after a median of 66, 57 and 138 tokens. The single loss V changed even more (96–100%, median first divergence 21–61 tokens) and was equally balanced (H1′: 48 new vs. 43 gone; H2′: 25 vs. 28).

H2 counts an unfinished answer as wrong, and most wrong answers were unfinished: 59 of G's 80 and 53 of L's 77 reached the 3,200-token limit without a boxed answer (29 new vs. 35 gone). Among answers that finished, 21 (G) and 24 (L) were wrong (13 new vs. 10 gone, $p = 0.68$); V finished more often (48 unfinished) and was wrong more often among finished answers (29; 17 new vs. 9 gone, $p = 0.17$). None of the secondary comparisons reached significance: broken copies 4 new vs. 3 gone, unfinished math 29 vs. 35, endless repetition in long-form answers 19 vs. 15 (all $p \geq 0.25$). The last comparison has new + gone = 34, below the 40 that the pre-registration requires for a verdict, so the one symptom with a detected tilt (Section 5.3) is also the one whose trajectory test is underpowered; its direction is positive. Accuracy on the 600 math problems was 86.7% (G) and 87.2% (L).

Table 3: Tilt at fragile steps of Qwen2.5-7B (972,380 output tokens). $\bar{\Delta}$: mean tilt in nats (Eq. 1); CI: 95% prompt-cluster bootstrap interval; control: the same quantity for the best normal alternative at the same steps; last row: steps at which llama.cpp itself chose b.

	O: other script	R: repetition	C: byte-split character
Fragile steps	3,042	4,975	2,800
per 1k tokens	3.13	5.12	2.88
$\bar{\Delta}_L$	-0.00006	$+\mathbf{0.0039}$	-0.0034
95% CI	$[-0.0046, +0.0046]$	$[+\mathbf{0.0015}, +\mathbf{0.0063}]$	$[-0.0074, +0.0004]$
Holm p	0.99	**0.003**	0.18
Label	no tilt	**tilts toward abnormal**	no tilt
Control $\bar{\Delta}_L$	$+0.0010$	-0.0011	$+0.008$ (103 steps)
$\bar{\Delta}_V$	-0.018	$+0.012$	$+0.012$
95% CI	$[-0.031, -0.006]$	$[+0.005, +0.018]$	$[+0.002, +0.023]$
L chose b	37 (1.2%)	101 (2.0%)	55 (2.0%)

Table 4: Symptom counts of Qwen2.5-7B under contract-exact computation (arm G).

Set	Items	S1 repetition	S2 other script	S6 broken copy
Natural: LCB Korean	396	6	65 (16%)	–
Natural: long-form Korean	300	95 (32%)	29	–
Trigger: rare syllables	200	10	44	115 (58%)
Trigger: mixed script	200	6	80 (40%)[a]	–
Trigger: repetition	200	52 (26%)	1	–

[a] 36 (18%) by the stricter judge of Section 5.6.

5.3 Tilt: only toward repetition, and small

The repetition tilt (Table 3) corresponds to a 0.39% increase in the odds of the repeating token; the control at the same steps is zero, so the shift is specific to repeating tokens rather than a general flattening. llama.cpp chose the abnormal token only where the margin was below 0.5 nat (below 0.25 nat: 36 of 206 O steps, 101 of 754 R steps, 52 of 314 C steps). V, whose activation rounding step is 2–4× llama.cpp's (Section 4.1), tilts about three times more toward repetition, also tilts toward byte errors (where llama.cpp shows no tilt), and tilts *away* from other-script tokens: different losses tilt in different directions, and the repetition tilt grows with the coarseness of activation rounding. This is consistent with activation rounding being one source of the repetition tilt, but V is not a controlled replica of llama.cpp's loss, so it does not isolate the source. On R1-Distill's English reasoning (8,121 repetition steps) neither L nor V tilts (-0.0006, -0.0009). Against fp64 arithmetic, the baseline's own rounding shows the same pattern at a smaller scale: $+0.0005$ nat toward repetition, none for the other types (Appendix E).

5.4 Under contract-exact computation and greedy decoding, the model produces the failures

R1-Distill answered 86.7% of the math problems correctly; 59 of the 80 wrong answers did not finish within 3,200 tokens. On the rare-syllable trigger the model sometimes loops on an emoji, or invents a Chinese "user" turn and repeats a character until the limit (Table 4). These rates are those of greedy decoding, which is known to favour repetition; with sampling they would differ. In real Korean text 11.2% (Qwen2.5) and 8.5% (Llama 3.1) of syllables are emitted across two or more byte-piece tokens; only 2,512 (Qwen2.5) and 599 (Llama 3.1) of the 11,172 Hangul syllables are single tokens.

5.5 Propagation and contamination

Forcing the abnormal token at an other-script fragile step makes more other-script text appear within the next 256 tokens in 80 of 100 continuations, against 34 after the original token (47 vs. 1 discordant, $p = 3.5 \times 10^{-13}$); 99 of the 100 forced tokens were complete characters, and excluding the first two characters of the continuation gives 77 of 99, so the effect is not a mechanical completion of a byte sequence. These steps are, by construction, contexts in which the other-script token was already among the top-20 candidates within $\ln 10$ of the chosen one; the comparison shows that in such contexts a single token changes the course of the text, not that an arbitrary slip would. For repetition and byte-split steps the measure saturated (the original continuations already showed the symptom in 89% and 97% of cases). A single anomaly injected into the previous answer did not raise symptoms in the next one: other-script text 32 vs. 37 (5 new, 10 gone), broken copies 53 vs. 54, repeated sentences 7 vs. 10.

5.6 Sensitivity to the symptom judge

A stricter judge, defined after the human check and before re-computation, ignores foreign text inside quotes or brackets or followed by a Hangul gloss, and ignores Latin words that occur in the prompt. It agrees with the human labels on 19 of 20 items (the items were used to design it, so this is not an independent check); a check on a fresh sample of 40 items is reported in Appendix F *[in progress]*. All conclusions hold: H1 31 new vs. 31 gone, H1′ 41 vs. 43, propagation 78% vs. 31%. Only the rates move: the mixed-script trigger's other-script rate falls from 40% to 18%; the LCB rate from 16% to 15%.

6 Discussion

No detectable degradation. At the precision of a mainstream engine with 8-bit weights, numerical loss behaves like re-sampling at near-ties: almost every long answer changes, and the failures it creates are matched, within our power, by failures it removes. The intervals leave room for a moderate increase—up to about a fifth of the other-script count—so the result is a bound, not a proof of absence. The harm we can see is reproducibility—the symptom status of about one item in ten (71 of 696, 53 of 600) depends on the engine—not quality.

The one tilt. The repetition tilt is real and specific, and the coarser activation rounding of V shows it three times more strongly, which points to activation rounding as one contributor without isolating it. Even the contract-exact baseline shows it against fp64 arithmetic, at +0.0005 nat on a 0.5B model (Appendix E): the tilt grows with the coarseness of rounding in the forward pass—contract (two digits) < llama.cpp (8-bit activations, half-precision attention) < V (7-bit activations). At 0.4% in odds it did not reach the trajectory level in our samples, and the repetition trajectory test was the least powered of all (new + gone = 34).

Under greedy decoding, the model is the main source. With contract-exact arithmetic the model loops on a third of long Korean answers, mixes scripts in a sixth of LCB answers and fails most rare-syllable copies; where an other-script token is already a close competitor, emitting it carries the text into that script most of the time. Mitigations for these failures belong to the model and the decoding procedure rather than to the arithmetic; greedy decoding favours repetition, and the base rates under sampling are not measured here.

What the baseline is and is not. It is contract-exact and deterministic, not exact: it rounds activations to two digits and evaluates the exponential as a polynomial. Its deviation from fp64 is three orders of magnitude below llama.cpp's on 0.5B (and, by simulation, on 7B); its own rounding, measured against fp64 on a 0.5B model (Appendix E), does not tilt other-script or

byte-split steps and tilts repetition steps by +0.0005 nat, one eighth of llama.cpp's tilt—so Δ_L in Section 5.3 understates llama.cpp's tilt relative to fp64 by about that much, if the two add. It made a 0.004-nat shift measurable and every case replayable bit for bit. The same result also bounds its practical value for users: effects small enough to need such a baseline are too small to change what users see.

Limitations. Two 7B models of one family at Q8_0, greedy decoding, one production engine (llama.cpp, default settings) on one GPU (RTX 4070 Ti), mostly Korean prompts, and only engine-level loss: we did not test larger losses such as 4-bit key–value caches, 4-bit weights (where Patodiya (2026) reports larger divergence), longer contexts or sampling. The judges are automatic; the other-script judge agreed with a human on 17 of 20 items, and the stricter judge was designed on those items. The follower reproduces one realization of llama.cpp's batch arithmetic. V is coarser than llama.cpp's activation quantization. Propagation could not be measured for repetition and byte-split steps because the measure saturated. Closed frontier models could not be studied; the mechanisms (byte pieces, near-ties, the pull of an emitted token) are generic, but their rates there are unknown.

7 Conclusion

With a contract-exact, noise-free baseline and identical inputs, the numerical loss of a production engine re-rolls LLM outputs without a detectable increase in their familiar failures—the net change is bounded within about ±20% of the baseline's symptom counts—and its only measurable bias is a 0.4% tilt toward repetition. Under greedy decoding the failures come from the model, and a single other-script token, where it was already plausible, carries the text along. At 8-bit precision, making inference arithmetic exact by contract buys reproducibility, not fewer failures.

Acknowledgements and AI assistance

The geul compiler, GeulPU and the experiment code were developed by the author with substantial assistance from AI coding agents (Anthropic Claude), which also helped draft this manuscript. The author designed the study, approved the pre-registration and checked the symptom judges by hand. A reader's critique of the first draft led to the interval bounds, the H2 split, the timeline of Section 4.6 and the wording of the title and conclusions.

References

[Draft: entries to be completed and checked.]

Bit-exact AI inference verification without performance tradeoffs. arXiv:2606.00279, 2026. [Authors to be added.]

T.-R. Chen. Deterministic LLM inference across GPU kernels: power-of-two INT8 quantization scales and the limits of tolerance-based conformance. arXiv:2609.00363, 2026.

K. Cobbe et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

L. Cooper et al. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. arXiv:2609.25624, 2026.

H. He et al. Defeating nondeterminism in LLM inference. Thinking Machines Lab, 2025.

S. Land and M. Bartolo. Fishing for Magikarp: automatically detecting under-trained tokens in large language models. arXiv:2405.05417, 2024.

K. Marchisio et al. Understanding and mitigating language confusion in LLMs. arXiv:2406.20052, 2024a.

K. Marchisio et al. How does quantization affect multilingual LLMs? arXiv:2407.03211, 2024b.

M. Nasr et al. Scalable extraction of training data from (production) language models. arXiv:2311.17035, 2023.

D. Pape et al. The silent hyperparameter: quantifying the impact of inference backends on LLM reproducibility. arXiv:2605.19537, 2026.

A. Patodiya. Same request, different answer: quantization amplifies cache-induced divergence in LLM serving. arXiv:2609.04748, 2026.

J. Qin et al. Vosti: specifying, implementing and verifying deterministic LLM inference. arXiv:2609.38981, 2026.

S. Woo. Raw data—GeulPU stage 40: engine numerical loss and LLM failure symptoms [data set]. Zenodo, 2026. https://doi.org/10.5281/zenodo.23417759

J. Xu et al. Learning to break the loop: analyzing and mitigating repetitions for neural text generation. arXiv:2206.02369, 2022.

J. Yuan et al. Give me FP32 or give me death? Challenges and solutions for reproducible reasoning. arXiv:2506.09501, 2025.

A Bit-identity evidence for GeulPU

Within one machine. Against the CPU reference: every logit row of Qwen2.5-0.5B; 26,155,008 logits of Qwen2.5-7B Q8_0 with zero differences; 106 kernel cases for Q4_0 and 349 for the k-quants plus whole files; the Qwen3, Llama 3.x and Mistral files end to end. Across execution paths: decode graph, whole prompt, prompt in four pieces, one row at a time and repeated runs give identical logits and key–value caches; wikitext-2 log-likelihoods are bit-identical whether text is fed in chunks of 512, 128 or 1 token (llama.cpp's perplexity moves between 15.8171 and 15.8200 with its batch settings); the server returns identical tokens and logit hashes for 4 models × 12 requests × 4 schedules (llama-server changed its greedy answer in 11 of 12 cases); lm-evaluation-harness item results are identical at 1 and 8 concurrent requests and on CUDA and Vulkan; Linux (WSL2) output equals Windows output byte for byte.

Across machines. A cross-check package builds the compiler, the back ends, the server and the 7B module from source on a volunteer's PC (the build hashes are compared with the reference), downloads DeepSeek-R1-Distill-Qwen-7B Q8_0, solves 32 synthetic math problems greedily ($\leq$3,200 tokens) and compares the texts and a 64-bit FNV hash of the full 152,064-entry logit row at every step (Table 5).

The 82,673 hashed steps are 82,657 generated tokens plus the 16 end-of-sequence steps. On the same reference PC, llama-server produced identical texts for 0 of the 32 problems between 1 and 8 concurrent requests and changed 6 final answers. An Intel UHD 770 integrated GPU reproduced the 0.5B model's records (larger models exceed its memory limits). Each run built the engine from the commit current at the time; later engine changes left the reference outputs unchanged and were re-verified on the reference PC only. The volunteers are acquaintances of the author running the published package, not independent replicators, and the 32 problems are synthetic.

Table 5: Cross-machine check. Texts: problems whose text is identical to the reference; logit hashes: steps whose logit-row hash is identical to the reference.

PC	GPU (path)	Driver	Texts	Logit hashes
Reference	NVIDIA RTX 4070 Ti, Ada (CUDA)	616.56	reference	reference
Volunteer 1	NVIDIA RTX 5070, Blackwell (CUDA)	596.21	32/32	82,673/82,673
Volunteer 2	NVIDIA RTX 3080 Ti, Ampere (CUDA)	616.92	32/32	82,673/82,673
Reference	RTX 4070 Ti (Vulkan; two kernel variants)	616.56	32/32	82,673/82,673
Volunteer 3	AMD RX 9060 XT, RDNA4 (Vulkan; two variants)	AMD 26.8.1	32/32	82,673/82,673
Volunteer 4	AMD RX 9060 XT (Vulkan; two variants)	AMD 25.20.42.14	32/32	82,673/82,673

B Deviations from the pre-registration

1. The 0.5B secondary model was dropped: the GeulPU server accepts only Qwen2-family models with head size 128.
2. The context was set to 4,096 tokens per slot (not specified in advance). Three LCB prompts too long to leave 256 new tokens were excluded from all arms; three others had their maximum length reduced to 808 tokens in all arms.
3. R1-Distill prompts were tokenized with the BOS token, as `llama-server`'s chat path does (the GGUF requests BOS; the template text does not contain it); the first tokenization without BOS was discarded before any measurement.
4. V's scoring of two very long LCB answers failed (context full); fragile steps whose tokens were outside V's top 20 (17, 1, 0) are missing from Δ_V.
5. Propagation for byte-split steps was measured only on the copy trigger, where a source text exists; propagation for repetition and byte-split steps saturated.
6. The misspelling injection failed in 8 contamination dialogs (fewer than three Hangul syllables in the middle sentence); those pairs are identical and do not contribute to the counts.
7. A selection bug (string comparison of item ids) emptied the determinism and follower-validation samples in the first run; the steps were re-run after the fix, without effect on any other number.
8. The other-script judge did not reach the pre-registered 90% agreement with the human check (Sections 5.1 and 5.6).
9. The prompt sets, judges and tool fixes were committed after the first run had started (Section 4.6), not before, although no comparison data existed yet. The interval bounds on the net change (Section 5.2), the split of H2 into unfinished and wrong-but-finished answers, and the "underpowered" reading of the long-form repetition comparison were added after a reader's critique of the first draft; the pre-registered labels are kept in the tables.
10. The registered expectation for Appendix E ("no tilt in all three types") was not met for repetition (+0.0005 nat, Holm $p = 0.0003$); the result is reported under the pre-registered labels.

C Reproduction and data

- **Pre-registration and record** (Korean): 문서/40단계.md in the repository—design (commit 9190f76, 09:28, before the first run at 09:51), prompt sets and judges (d88b6cf, 10:00, during G's Qwen2.5 run), tool fixes (eedb304, 10:44, after that run and before the V and L runs), results (727a218), human check and the strict-judge analysis (0a5ab97).
- **Prompt sets**: 시험/질문40/ with sources, licenses (LCB: Apache-2.0; gsm8k: MIT) and SHA-256 hashes in 시험/질문40/출처.md; built by 재기/질문40.py (seed 40).
- **Tools**: 재기/증상40.py (symptom and fragile-step judges, statistics; 93 unit tests), 재기/모음40.py (tokenization through llama.cpp, generation in the three arms, V's scoring, contamination, forced-token continuations, determinism re-runs), 재기/따라40.py (follower of llama.cpp's distributions through its C API), 재기/셈40.py (analysis), 재기/사후40.py (strict judge; 12 unit tests), 공식실행/시험40.sh (the order of steps).
- **Command**: bash 공식실행/시험40.sh, given the repository, the work directory, an ASCII-path model directory and the llama.cpp directory, runs build → tokenization → G → contamination → V → L → follower → checks → fragile steps → forced-token continuations → determinism → analysis.
- **Versions and hashes** (문서/결과/40단계/40_기록.txt): GeulPU server 010f5dae..., kernel generator a304a884..., modules qwen2Q8_7b and qwen2Q8_7b_자리하나 equal to the repository's module-hash table; llama.cpp b11669 (commit 620d2225c, Windows CUDA 13.4 build); models Qwen2.5-7B-Instruct Q8_0 (SHA-256 52c5bf14...) and DeepSeek-R1-Distill-Qwen-7B Q8_0 (7307bfbb...); NVIDIA RTX 4070 Ti, driver 616.56.
- **Results**: 문서/결과/40단계/40_판정.json (all numbers), 40_사후_S2엄격.json (strict judge), 사람확인.md (human check).
- **Raw data**: Zenodo, DOI 10.5281/zenodo.23417759 (Woo, 2026)—five zip archives (1.7 GB compressed, 4.3 GB unpacked): every generated token with its top-20 log-probabilities for G, V and L, the follower's per-step log-probabilities, the fragile steps, forced-token continuations, contamination dialogs, the token ids sent, and the engine executables and kernels used, with a README describing the formats and SHA-256 manifests. Every case generated by G can be replayed bit for bit on any supported GPU or on the CPU reference.

D The geul language, the compiler and the engine

D.1 The geul language and its compiler

geul (글, "writing") is a systems programming language whose grammar follows Korean: verbs come last, and the role of each argument is marked by a case particle instead of its position. A function is declared with its roles, [문자열 원본을 문자열 대상에 복사] ("copy a string *source* to a string *destination*"), and called by role, 원본을 대상에 복사하다.; the particles 을/를 (object), 에 (destination), 에서 (source), 로 (means) and 와/과 (companion) bind the arguments, and a call whose same-typed arguments carry no particle is a compile error. Types are ordinary—정수 is a 64-bit integer, 짧은실수 a 32-bit float, T 참조 a pointer—and memory is managed by hand.

The compiler is self-hosted: it is written in geul (about 10,000 lines) and compiles itself. A separate reference implementation in Python (about 7,000 lines) is kept as the bootstrap seed and as a second implementation; the build checks a fixpoint—the self-hosted compiler built by the reference implementation, then by itself, then by that output, is byte-identical each time. Users need neither: the released compiler geulc.exe (geul 1.3.1, October 2026, MIT license; a 182 KB zip, also packaged for Windows as the PyPI package geul) builds everything described below. The compiler has its own complete back end—lexer, parser, semantic analysis, a typed intermediate representation, an inliner, an x86-64 code generator with linear-scan register allocation, a hand-written instruction encoder and PE32+/ELF64 writers—and uses no C compiler, assembler, linker or LLVM. A Windows executable it produces imports only kernel32.dll

(and `shell32.dll` for the command line); the Linux target (on the research branch) needs only `libc.so.6`. Builds are reproducible byte for byte.

D.2 From geul source to GPU instructions

GPU support lives on a research branch of the compiler. A GPU kernel is an ordinary geul function; GPU features are reached through declared intrinsics: thread and block indices, shared memory, barriers, warp shuffles, asynchronous copies (`cp.async`), integer tensor-core matrix multiply–accumulate (`mma.sync` ... `m16n8k32` ... `s8.s8.s32` and `m16n8k16`), `dp4a`, and fused multiply–add, which is emitted only where the source writes it. Two back ends, both written in geul on top of the self-hosted front end, translate such kernels:

- **PTX back end** (2,574 lines)—emits PTX, NVIDIA's documented virtual instruction set. Every floating-point operation carries an explicit rounding mode (`.rn`), so the driver cannot fuse a multiply and an add on its own; approximate instructions and floating-point tensor cores, whose summation rounding the PTX specification leaves open, are not used.
- **SPIR-V back end** (4,006 lines)—emits SPIR-V 1.6 compute shaders for Vulkan (AMD, Intel and NVIDIA). Kernel parameters become push constants and global memory becomes 64-bit device addresses; division and square root are written out to be correctly rounded, integer matrix products use cooperative-matrix instructions where the device offers them and subgroup operations otherwise.

At run time the engine hands PTX text to the NVIDIA driver (`cuModuleLoadData`), whose built-in just-in-time compiler translates it to the GPU's native machine code (SASS), and launches kernels with `cuLaunchKernel` and CUDA graphs (Figure 1). On Vulkan it creates shader modules and compute pipelines and records dispatches; a thin layer maps the driver-API calls the engine uses onto Vulkan, so the engine code is shared. Nothing from the CUDA toolkit—`nvcc`, `ptxas`, `cudart`, cuBLAS—or from a Vulkan SDK is used to build or run the engine; the continuous-integration job installs `ptxas` only to check register spills. This is the same layer at which other engines meet the GPU: llama.cpp ships kernels compiled ahead of time with `nvcc` and calls cuBLAS, vLLM runs PyTorch, CUDA and Triton kernels; GeulPU generates its own kernels and talks only to the driver. The vendors' closed compilers below PTX and SPIR-V are outside the contract, which is why bit identity across devices is established empirically (Appendix A) rather than proved.

D.3 The kernel generator and the engine

A kernel generator, itself a geul program (8,056 lines), writes the kernel source specialized to a model's shape and weight format. For Qwen2.5-7B at Q8_0 it writes 141,818 lines of geul, which the PTX back end turns into 73 MB of PTX holding 106 kernels and 6,488 tensor-core instructions; the module is built on the user's machine on first use, and the CI regenerates 18 such modules and compares their SHA-256 hashes with the recorded ones.

The host—about 13,000 lines of geul (15,000 with its test programs)—is a complete serving engine:

- **Models**: reads GGUF files with Q8_0, Q4_0, Q4_K, Q5_K and Q6_K weights (including the Q4_K_M and Q5_K_M mixes); architectures qwen2, qwen3 and llama—Qwen2.5, Qwen3, DeepSeek-R1-Distill-Qwen, Llama 3.1 and 3.2, Mistral v0.3—with head size 128 (Qwen2.5-0.5B only through the command-line runner).
- **Text**: byte-level BPE with Unicode NFC, the Llama 3 BPE and SentencePiece tokenizers, written in geul, which match llama.cpp token for token on 2,000 lines of test text; chat templates checked against the GGUF's Jinja templates.

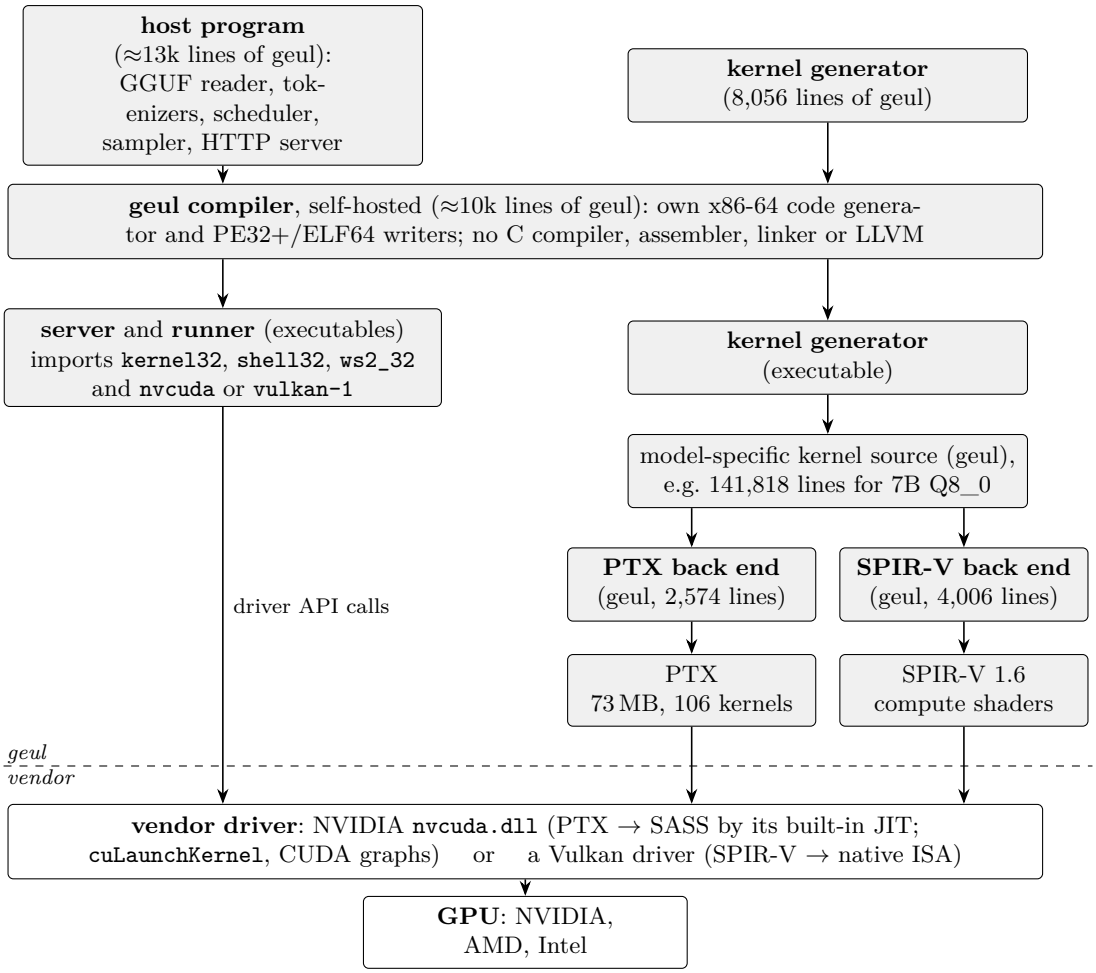


Figure 1: The GeulPU stack. Everything above the dashed line is written in geul and compiled by the geul compiler; below it is the GPU vendor's driver.

- **Serving**: an OpenAI-compatible HTTP server (chat and text completions, streaming, `/tokenize`), up to 8 slots with continuous batching and per-slot prompt reuse, a decode step captured as one CUDA graph, sampling and log-probabilities computed under the same written contract.
- **Footprint**: the CUDA server imports only `kernel32`, `shell32`, `ws2_32` and `nvcuda` (43 driver-API functions); the Vulkan server imports `vulkan-1` (48 functions) instead of `nvcuda`. The CI checks these import tables.

On an RTX 4070 Ti, its decoding speed is within ±5% of llama.cpp for Qwen2.5-7B and Llama-3.1-8B at Q8_0 and Q4_K_M (0.97–1.05× the time per token, depending on the llama.cpp build and the context length); prompt processing is faster than llama.cpp below about 128 tokens and up to 1.3–1.4× (Q8_0) and 1.4–1.6× (k-quant formats) slower at 8,192 tokens. Not everything is in geul: the CPU reference of the contract, the test harnesses and the benchmarks are Python (NumPy), and the optional desktop app's interface is JavaScript. The engine is Windows-first (Linux builds were tested under WSL2), uses one GPU, and needs compute capability 8.0 or newer on the CUDA path.

E Does the baseline itself tilt? (Qwen2.5-0.5B against fp64)

The contract-exact baseline rounds activations to two digits and evaluates the exponential as a polynomial. To measure whether this rounding itself tilts fragile steps, we used Qwen2.5-0.5B-

Instruct (Q8_0)—the largest model whose fp64 weights (4 GB) fit the CPU—and the 1,296 Korean answers of G (Qwen2.5-7B) as prefixes: the arithmetic question does not depend on whose text the prefix is, and the 0.5B model's own Korean generations loop too often to give natural contexts. The design was registered before the measurement (`문서/40단계.md`, "40g", commit `73b1497`).

- **G-0.5B**: GeulPU's engine (the same kernels as the server) computed the contract-exact distribution at every step of the 972,380 answer tokens, 512 rows at a time; the logits of this path equal those of the generation path bit for bit (checked on the first item). Its top-1 equals the 7B text's next token at 82.9% of steps.
- **fp64**: the same model computed in NumPy float64 from the GGUF weights ($q \cdot d$ exactly)—RMSNorm, RoPE, GQA attention, SwiGLU and the output projection all in fp64, the computation of `시험/참값.py`. Its top-1 equals G-0.5B's at 99.96% of the 7,325 steps of the first 20 items.
- **Fragile steps** were selected with the pre-registered judge from G-0.5B's top 20, at steps where its top-1 equals the next token of the text (the judge's condition): O 4,070, R 1,730, C 5,155 (4.19, 1.78 and 5.30 per 1k tokens). $\Delta_G = [\ln p_G(b) - \ln p_G(a)] - [\ln p_{64}(b) - \ln p_{64}(a)]$; statistics and labels as in Section 4.4.

Table 6: Tilt of the contract-exact baseline against fp64 arithmetic (Qwen2.5-0.5B on the 7B answers; 972,380 steps).

Type	Steps	$\bar{\Delta}_G$, 95% CI	Holm p	Label	Control $\bar{\Delta}_G$	fp64 argmax $\neq a$
O other script	4,070	-0.0027 $[-0.0066, +0.0001]$	0.22	no tilt	-0.0020	5
R repetition	1,730	$\mathbf{+0.0005}$ $[\mathbf{+0.0002}, \mathbf{+0.0008}]$	**0.0003**	**tilts toward abnormal**	-0.0002 ($p = 0.15$)	4
C byte-split	5,155	-0.0003 $[-0.0008, +0.0001]$	0.22	no tilt	-0.0002 (167 steps)	5

The contract's rounding does not tilt other-script or byte-split steps (intervals within ± 0.007 nat) and tilts repetition steps toward the repeating token by $+0.0005$ nat—about 0.05% in odds, one eighth of llama.cpp's $+0.0039$ on 7B and one twenty-fourth of V's $+0.012$—with the control at the same steps at zero (Table 6). The registered expectation (no tilt in all three types) was therefore not met for repetition: the measurement is precise enough to detect a tilt that small. Two cautions: selecting fragile steps by G's own margin (a threshold on a noisy quantity) can bias Δ_G upward for any abnormal candidate, and the O and C types, selected the same way, show no such bias, so it is below 0.001 nat and the effect is specific to repetition; and the model and prefixes differ from the main study. The ordering contract (two digits) < llama.cpp (8-bit activations, half-precision attention) < V (7-bit activations) supports reading the repetition tilt as a generic effect of rounding in the forward pass that grows with its coarseness. The baseline is thus not perfectly neutral at repetition steps; its own tilt is an order of magnitude below the effect it was used to measure, so Δ_L in Section 5.3 understates llama.cpp's tilt relative to fp64 by about that amount if the two add. Tools: `재기/기준40.py`; numbers in `문서/결과/40단계/40g_기준기울기.json`.

F Second human check of the strict judge

[In progress: 40 fresh items (20 judged positive and 20 negative by the strict judge of Section 5.6, disjoint from the 20 used to design it) labelled by hand.]